

LAB ASSIGNMENT 2

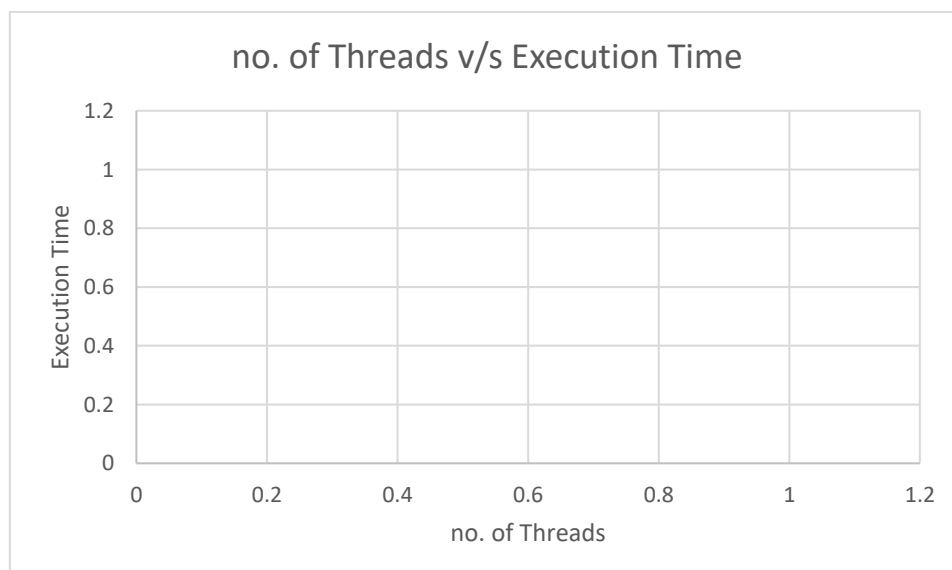
UCS645

Q1 Molecular Dynamics

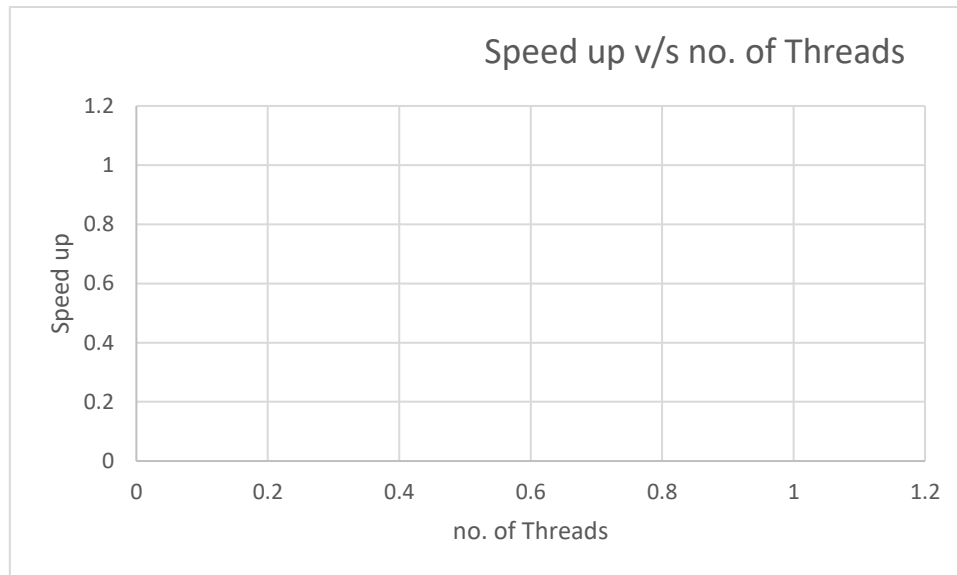
Observed Result

No. of Threads	Execution Time	Total Energy	Speed up	Efficiency	Performance
2	0.0079999	3.18578e+28	1.00	50%	Baseline execution with minimal parallel overhead
4	0.00500011	3.18578e+28	1.60	40%	Effective utilization of additional cores
8	0.00399995	3.18578e+28	2.00	25%	Best performance due to optimal core usage
16	0.0059998	3.18578e+28	1.33	8%	Performance degradation due to synchronisation and oversubscription

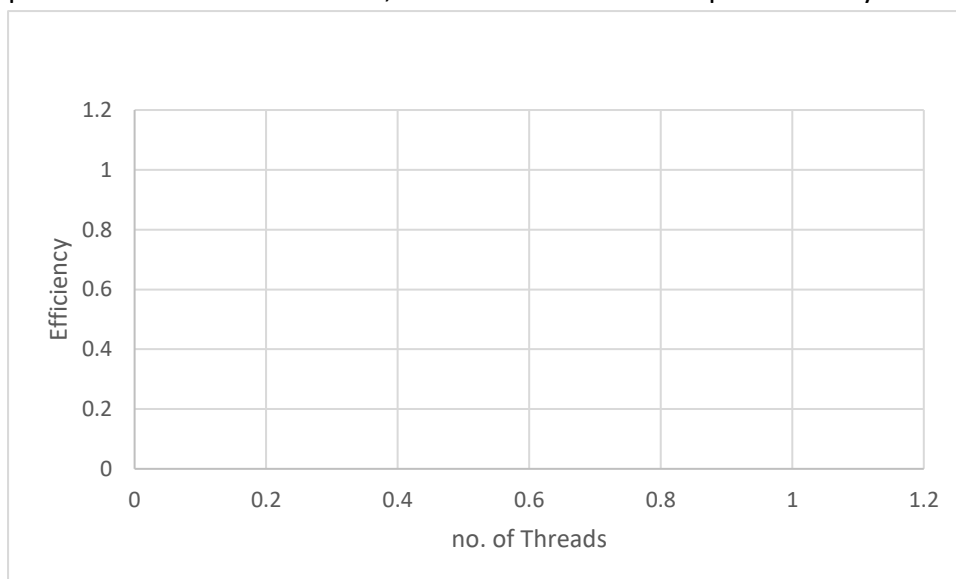
The molecular dynamics force calculation scales efficiently up to 8 threads, after which synchronization overhead and memory bandwidth limitations reduce parallel efficiency.



The execution time decreases significantly as the number of threads increases from 2 to 8, indicating effective parallelization and improved resource utilization. The minimum execution time occurs at 8 threads, representing the optimal performance point. Beyond this, execution time increases at 16 threads due to synchronization overhead, thread contention, and memory bandwidth saturation. This confirms that adding threads beyond the optimal core count leads to diminishing and eventually negative performance gains.



The speedup increases with the number of threads up to 8 threads, indicating effective parallel utilization of computational resources. Beyond this point, the speedup declines at 16 threads due to increased synchronization overhead, thread contention, and memory bandwidth limitations. This demonstrates that the molecular dynamics implementation achieves optimal performance near 8 threads, after which additional parallelism yields diminishing returns.



The parallel efficiency decreases steadily as the number of threads increases, indicating rising overhead and resource contention. The system achieves the best utilisation at lower thread counts (around 2–8 threads), while efficiency drops sharply at 16 threads due to

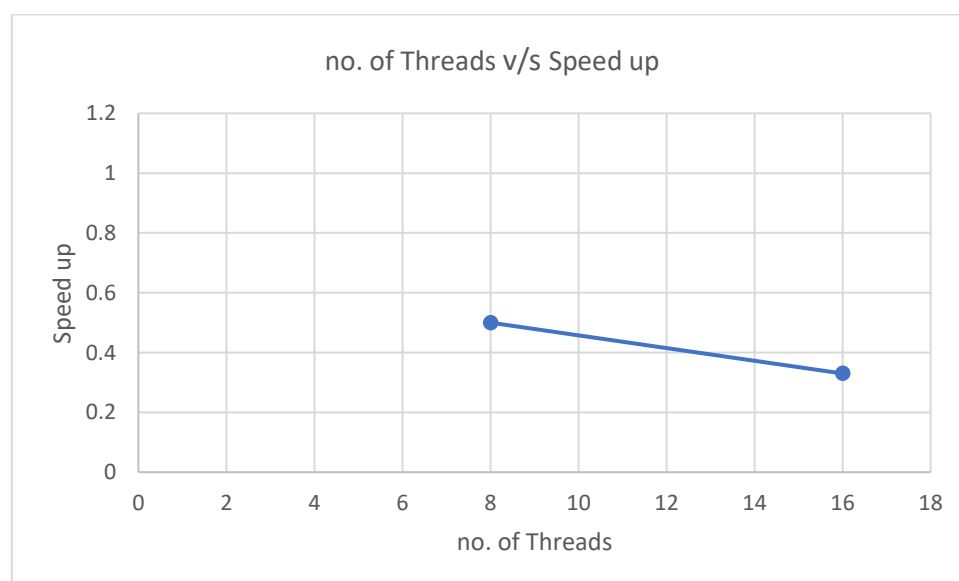
synchronisation costs and memory bandwidth limitations. This confirms that the molecular dynamics force calculation has an optimal parallel range beyond which additional threads provide diminishing returns.

Q2 Bioinformatics - DNA Sequence Alignment

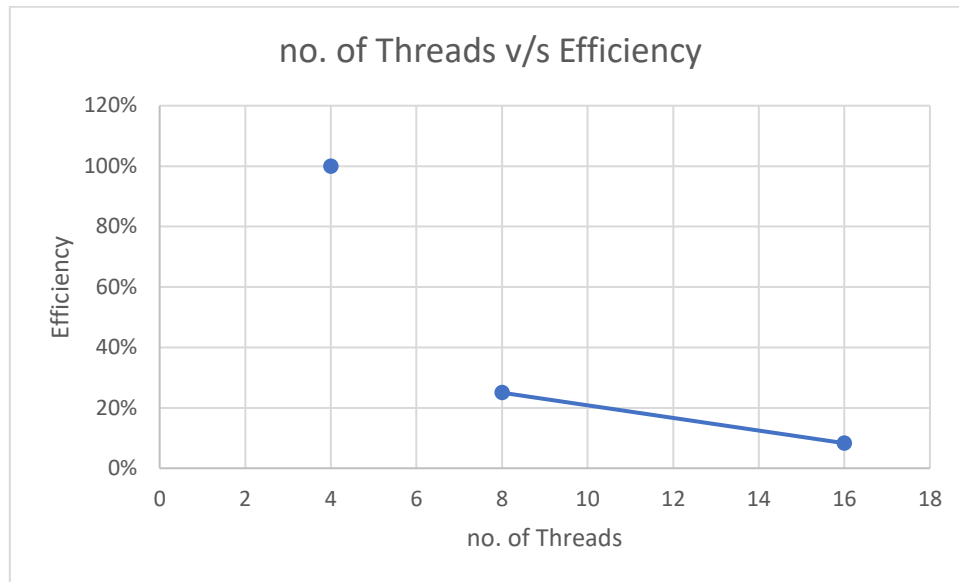
Observed Result

No. of Threads	Execution Time	Speed up	Efficiency	Performance
2	0.0079999	-	-	Time too small to measure accurately
4	0.00100017	1.00	100%	Baseline measurable case
8	0.00199986	0.5	25%	Overhead dominates
16	0.00300002	0.33	8.3%	Performance degrades due to overhead and limited parallelism

The parallel Smith–Waterman implementation produces correct alignment scores across all thread counts; however, performance does not improve with additional threads due to the small problem size and inherent synchronization overhead of wavefront parallelization. The results indicate that for short DNA sequences, parallel overhead outweighs computational benefits, leading to limited scalability.



The speedup decreases as the number of threads increases, indicating poor parallel scalability of the Smith–Waterman implementation for the given problem size. This behavior occurs because wavefront synchronization overhead and limited parallelism dominate the computation. The results confirm that for small DNA sequences, additional threads do not improve performance and instead introduce overhead, leading to diminishing returns.



The parallel efficiency drops sharply as the number of threads increases, indicating significant overhead and limited effective parallelism in the Smith–Waterman implementation. While the baseline (4 threads) shows full efficiency, additional threads lead to underutilization due to synchronization costs and the dependency-constrained wavefront computation. This confirms that the algorithm does not scale efficiently for the given small problem size.

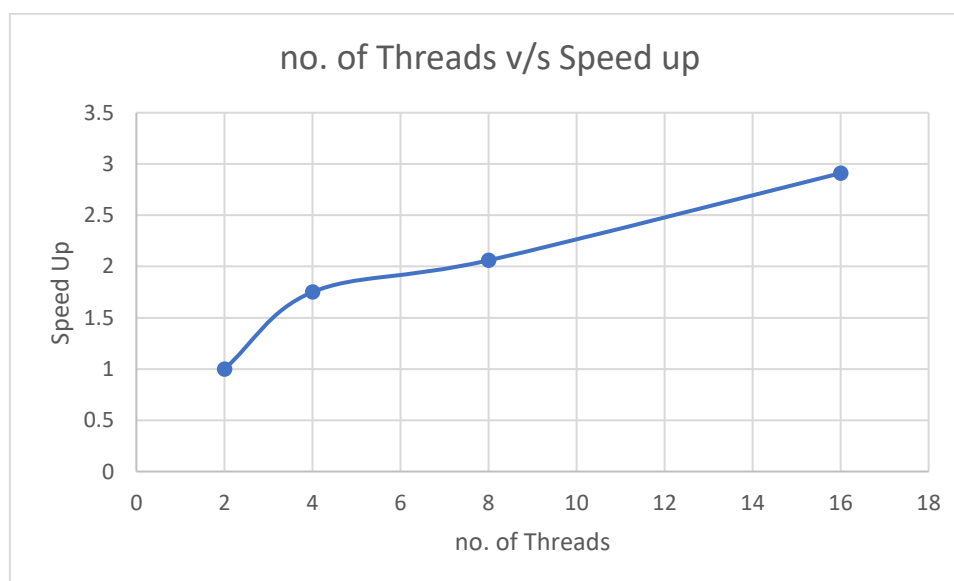
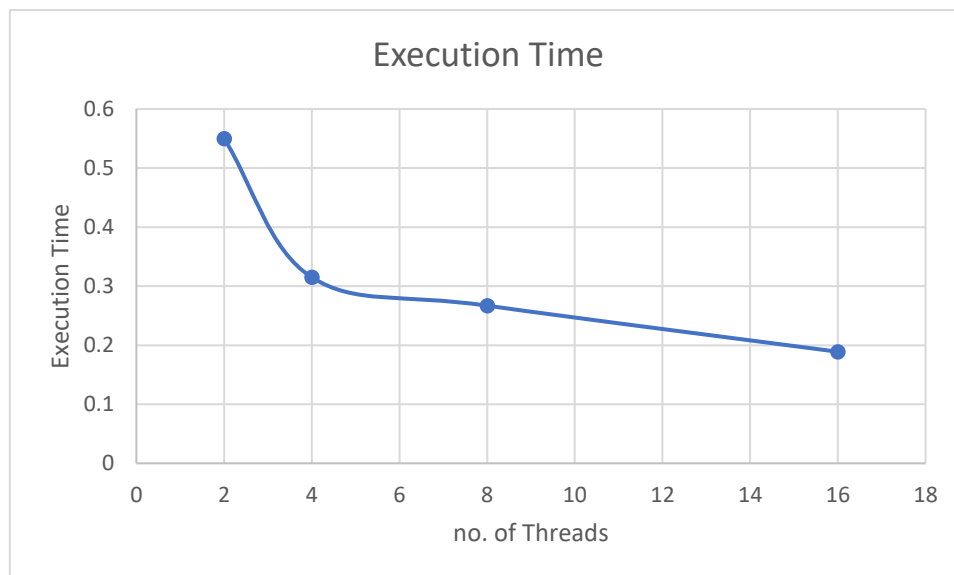
Q3 Scientific Computing - Heat Diffusion Simulation

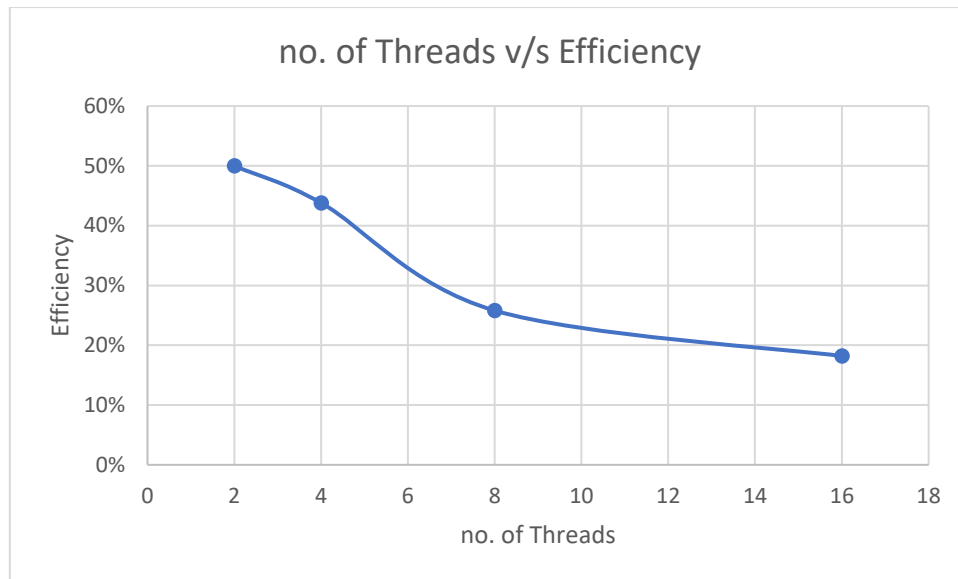
Observed Results

No. of Threads	Execution Time	Total Energy	Speed up	Efficiency	Performance
2	0.55	0.159235	1.00	50%	Baseline parallel execution
4	0.315	0.0795975	1.75	43.8%	Good scaling due to spatial parallelism
8	0.267	0.0530605	2.06	25.8%	Moderate improvement;

					memory effects begin
16	0.189	0.0397937	2.91	18.2%	Continued speedup but reduced efficiency

The heat diffusion simulation demonstrates good parallel scalability, with execution time decreasing steadily as the number of threads increases. The best performance is observed at 16 threads, although parallel efficiency gradually declines due to memory bandwidth limitations and parallel overhead. Overall, the stencil-based computation benefits significantly from spatial parallelization and exhibits better scalability compared to dependency-heavy algorithms.





The heat diffusion simulation exhibits strong parallel scalability, with execution time decreasing steadily and speedup increasing up to 16 threads, demonstrating effective spatial parallelization. However, parallel efficiency declines gradually as thread count grows, indicating increasing memory bandwidth pressure and parallel overhead. Overall, the algorithm scales well on multicore systems but shows the expected efficiency loss at higher thread counts due to hardware and memory limitations.
